

Manual de uso de las DevTools del navegador

Natalia Cabello Luque
2ºDaw
Desarrollo Entorno Cliente
25/09/2026

ÍNDICE

1.Introducción

2.Descripción del código de ejemplo

2.1.HTML

2.2.CSS

2.3.JS

3.Pestaña Elements

3.1 Inspección del DOM

3.2 Modificación de estilos

3.3 Panel Computed

3.4 Edición de atributos y clases

4.Pestaña Console

4.1 Logs y errores

4.2 Uso de comandos

4.3 Filtrado de mensajes

5.Pestaña Sources (Depuración)

5.1 Breakpoints

5.2 Step over / Step into / Step out

5.3 Panel Scope

5.4 Watch expressions

5.5 Pausa en excepciones

6.Pestaña Network

6.1 Carga de recursos

6.2 Tiempos y waterfall

6.3 Cabeceras y respuestas

7.Pestaña Application

7.1 Cookies

7.2 LocalStorage

7.3 SessionStorage

8.Pestaña Performance

8.1 Grabación de rendimiento

8.2 Interpretación de métricas

9.Pestaña Lighthouse

9.1 Auditoría

9.2 Resultados y recomendaciones

10.Conclusiones

11.ENLACE A GITHUB

1.Introducción

Las **DevTools** de los navegadores modernos constituyen un conjunto indispensable de herramientas de inspección, depuración y análisis para cualquier desarrollador web. Su propósito es permitir una comprensión profunda del comportamiento de una aplicación en tiempo real, facilitando la identificación de errores, la optimización del rendimiento y la verificación del correcto funcionamiento del código. Se accede a ellas mediante **F12, Ctrl + Shift + I** o **Cmd + Option + I** en macOS.

El objetivo de este manual es ofrecer una guía clara, estructurada y práctica para aprender a utilizar las principales pestañas de las DevTools. A través de ejemplos reales y explicaciones detalladas, el lector podrá comprender cómo inspeccionar el DOM, depurar código JavaScript, analizar peticiones de red, gestionar el almacenamiento local y evaluar el rendimiento de una aplicación web.

2.Descripción del código de ejemplo

Para la elaboración de este manual se ha desarrollado una pequeña aplicación web compuesta por **HTML, CSS y JavaScript**. El objetivo de esta aplicación es servir como un banco de pruebas para el uso de las DevTools del navegador.

2.1.HTML

Calculadora de descuento

Resultado: -

Calcular descuento

Provocar error

```
index.html M
practicas > practica1 > index.html > ...
2  <html lang="en">
3
4  <head>
5    <meta charset="UTF-8">
6    <meta name="viewport" content="width=device-width, initial-scale=1.0">
7    <link rel="stylesheet" href="./stylos/stylos.css">
8    <script src="./js/practica1.js" defer></script>
9    <script src=""></script>
10   <title>PRACTICA 1 DevTools EN EL NAVEGADOR</title>
11 </head>
12
13 <body>
14   <!-- Titulo Principal del programa -->
15   <h1 id="titulo">Calculadora de descuento</h1>
16   <!-- Mostrar resultado -->
17   <p id="resultado">Resultado: -</p>
18
19   <!-- Botones para los calculos -->
20   <button id="btnCalcular">Calcular descuento</button>
21   <button id="btnError">Provocar error</button>
22
23 </body>
24
25 </html>
26
```

2.2.CSS

```
# styles.css
practicas > practica1 > styles > # styles.css > ...
    Usted, hace 4 minutos | autores (Natalialuque y otro más)
1  /* Estilos generales */
2  body {
3    font-family: Arial, Helvetica, sans-serif;
4    text-align: center;
5    margin-top: 50px;
6  }
7
8  /* Estilos del titulo */
9  #titulo{
10   color: rgb(10, 10, 98);
11 }
12
13 /* Estilos boton */
14 button {
15   padding: 10px;
16   margin: 5px;
17 }
18
19 /* Estilos del resultado */
20 #resultado {
21   font-size: 20px;
22   color: rgb(156, 240, 156);
23 }
24
25
```

2.3.JS

En esta imagen se muestran las funciones principales de la aplicación y la relación entre que existe entre ellas, con esta parte vamos a conseguir poder probar diferentes cosas de este manual.

```
JS practica1.js
practicas > practica1 > js > JS practica1.js > iniciarCalculo
    Natalialuque, hace 22 horas | autores (Usted y otro más)
1  /**Variables de la clase index.html */
2  const btnCalcular = document.getElementById("btnCalcular");
3  const btnError = document.getElementById("btnError");
4  let resultados = document.getElementById("resultado");
5
6  /**Lo que hace el boton al clicarlo */
7  btnCalcular.addEventListener("click", iniciarCalculo);
8  btnError.addEventListener("click", provocarError);
9
10 /**Funcion de iniciar calculo donde tenemos el precio, varios console.log
11  * y llamamos a otra funcion para hacer concatenacion de ellas */
12 function iniciarCalculo() {
13   let precio = 200;
14
15   console.log("Iniciando cálculo del descuento");
16
17   console.warn(
18     "Se va a utilizar un cálculo con un error de lógica intencionado",
19   );
20
21   console.table({
22     // Natalialuque, hace 22 horas * practica 1 terminada
23     {
24       precioOriginal: precio,
25       descuento: "20%",
26     },
27   });
28
29   let resultado = calcularDescuento(precio);
30
31   resultados.innerHTML= resultado + " €";
32 }
```

```
/**Funcion para realizar el descuento */
function calcularDescuento(precio) {
  let descuento = obtenerPorcentaje();
  // Error de lógica intencionado
  let precioFinal = precio - descuento;
  return precioFinal;
}

/**Funcion para obtención del porcentaje */
function obtenerPorcentaje() {
  let porcentaje = 20;
  for (let i = 0; i < 3; i++) {
    console.log("Iteración:", i);
  }
  return porcentaje;
}

/**Funcion para poder provocar un error */
function provocarError() {
  throw new Error("Error provocado para la práctica");
}
```

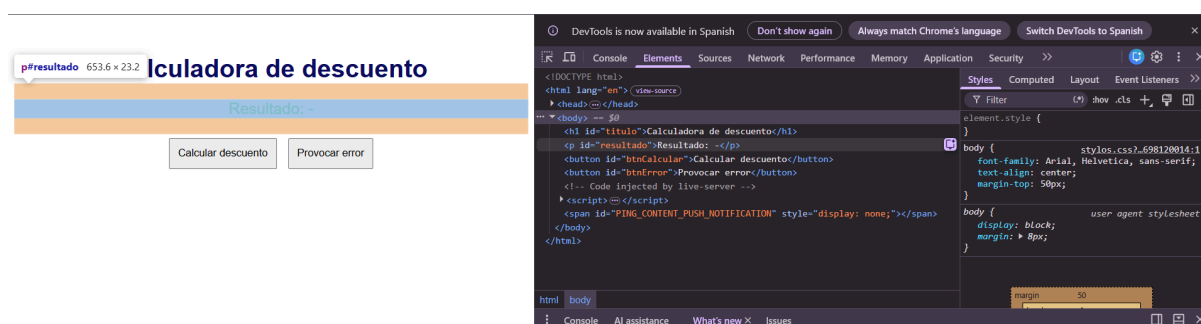
3. Pestaña Elements

La pestaña **Elements** permite inspeccionar y modificar en tiempo real la estructura HTML de una página web y los estilos CSS que tienen aplicados sus elementos. Además, ofrece herramientas para visualizar los estilos calculados por el navegador y para modificar atributos o clases sin necesidad de editar el código fuente directamente.

3.1 Inspección del DOM

Esta herramienta permite visualizar la estructura HTML de la página y localizar cada uno de los elementos que la componen.

En esta captura puede observarse cómo la herramienta **Elements** permite localizar visualmente cualquier elemento de la página.



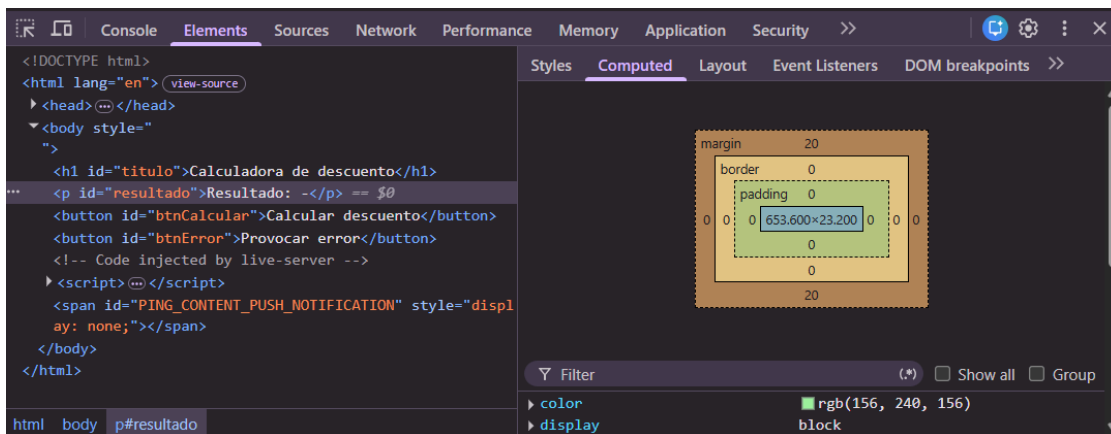
3.2 Modificación de estilos

Una de las características más útiles de **Elements** es la posibilidad de modificar los estilos CSS en tiempo real.



3.3 Panel Computed

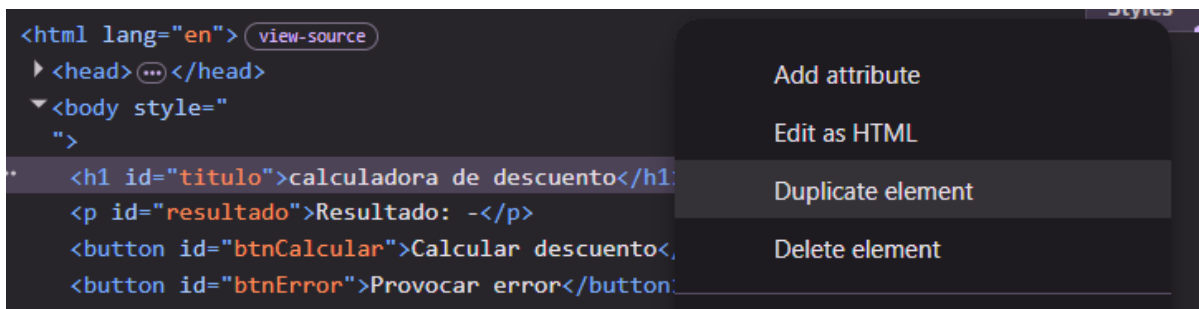
El panel **Computed** muestra los estilos finales que el navegador aplica a un elemento después de resolver todas las reglas CSS activas.



3.4 Edición de atributos y clases

Además de modificar estilos, la pestaña Elements permite editar atributos HTML y clases CSS directamente desde el DOM.

Y desde aquí te permite editarlo directamente.



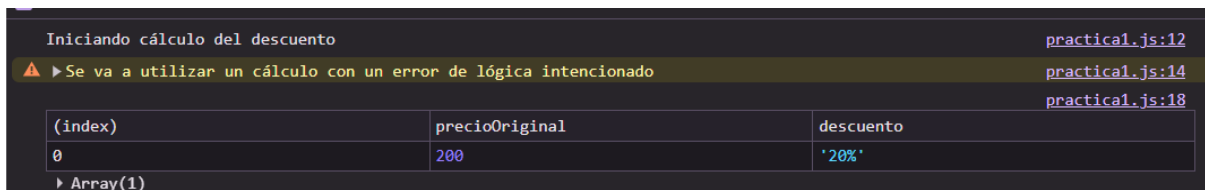
4. Pestaña Console

La pestaña **Console** permite visualizar mensajes generados por el código JavaScript, detectar errores, ejecutar instrucciones directamente desde el navegador y filtrar la información mostrada. Se trata de una de las herramientas más utilizadas durante el desarrollo y la depuración de aplicaciones web.

4.1 Logs y errores

La consola permite mostrar información mediante diferentes métodos, cada uno con una finalidad distinta. En la aplicación desarrollada para esta práctica se utilizan los métodos `console.log()`, `console.warn()`, `console.table()` y un error provocado mediante `throw new Error()`.

En primer lugar vamos a ver como se muestran los **log**, al pulsar nuestro botón de calcular descuento:

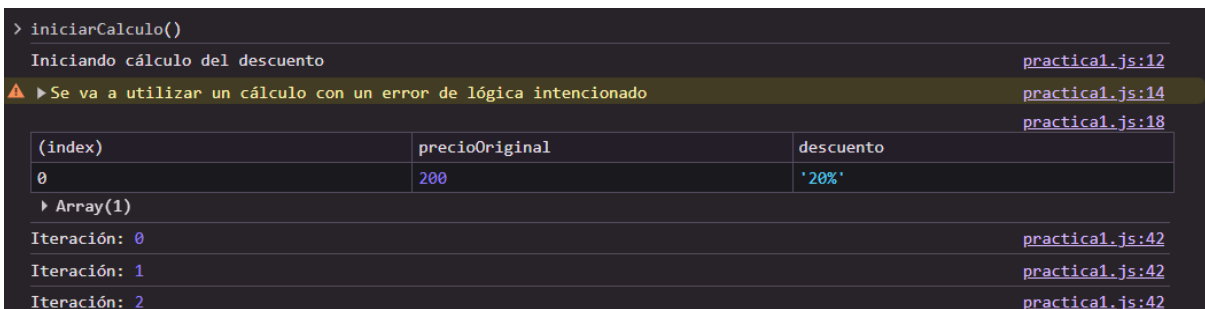


Y ahora vamos a ver el `Throw new Error()`, si le pulsamos nos dirige a la línea :



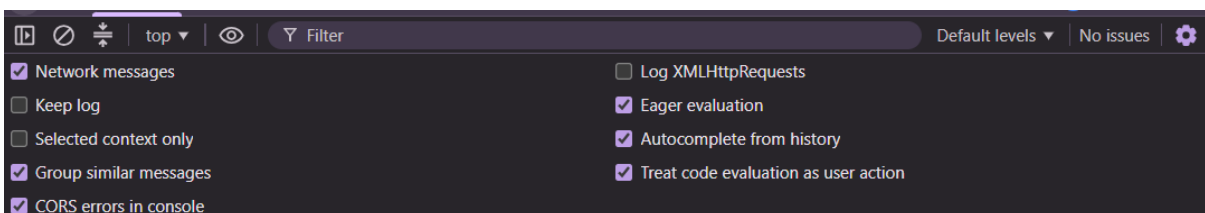
4.2 Uso de comandos

Además de visualizar mensajes, la consola permite ejecutar código JavaScript directamente sobre la página cargada. En este caso probamos con la función de `iniciarCalculo()`, la cual nos muestra lo mismo que si pulsamos el botón



4.3 Filtrado de mensajes

Cuando una aplicación genera muchos mensajes, la consola puede resultar difícil de interpretar. Para facilitar el análisis, DevTools incorpora herramientas de filtrado.

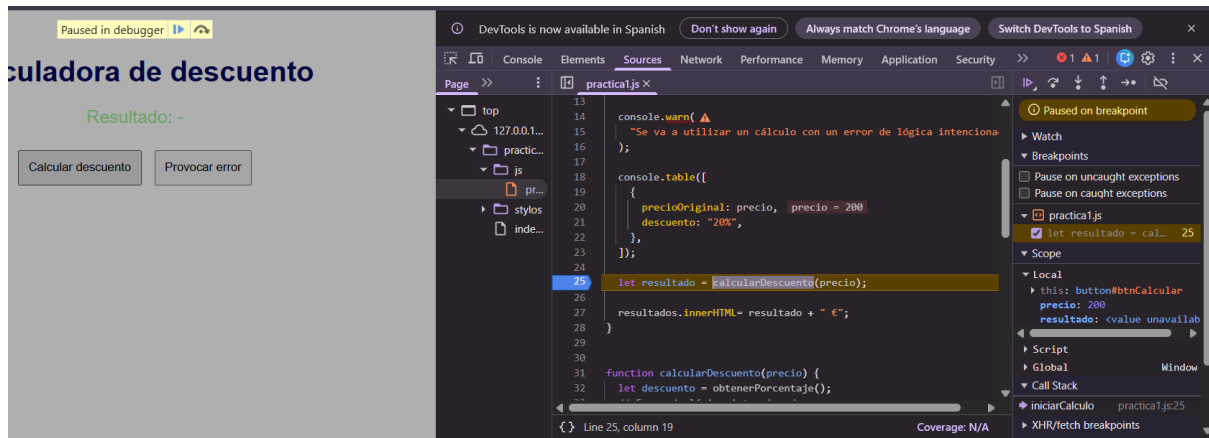


5. Pestaña Sources (Depuración)

La pestaña **Sources** es una de las herramientas más importantes de las DevTools, ya que permite analizar el funcionamiento interno del código JavaScript mientras se está ejecutando. Desde este panel es posible establecer puntos de interrupción, ejecutar el código paso a paso, inspeccionar variables y detectar errores de ejecución.

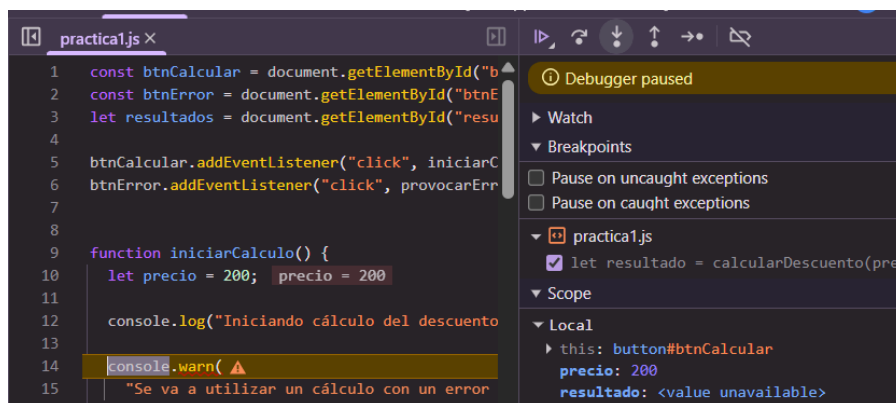
5.1 Breakpoints

Permiten detener temporalmente la ejecución de un programa en una línea concreta de código para analizar su comportamiento.

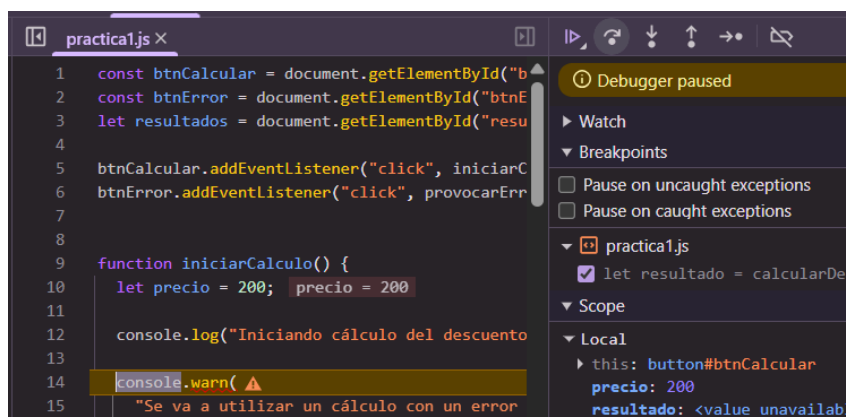


5.2 Step over / Step into / Step out

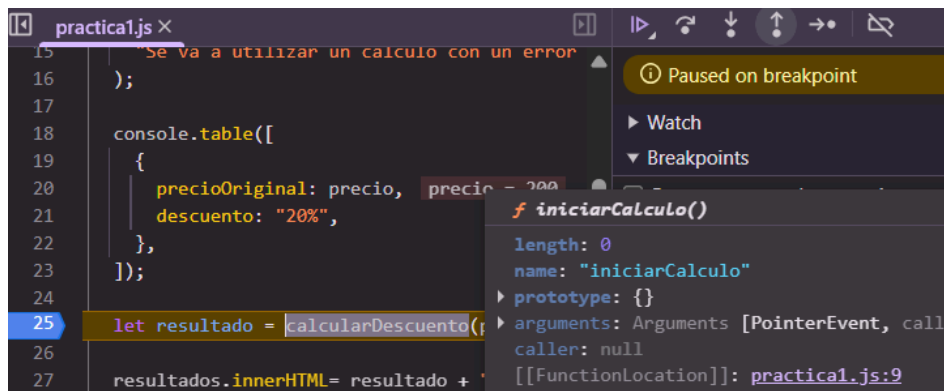
Step Into permite analizar el funcionamiento interno de una función línea a línea.



Step Over ejecuta la línea actual sin entrar en posibles funciones que se invoquen en ella.

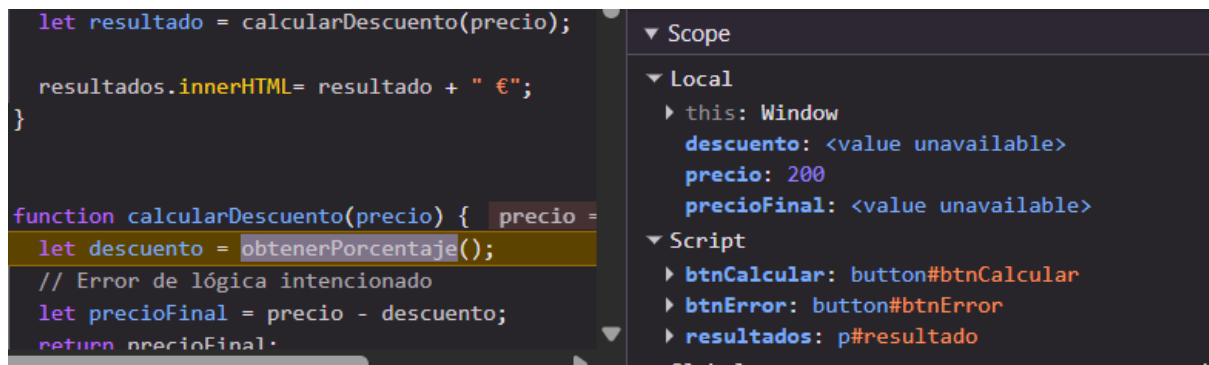


Step Out permite abandonar rápidamente una función cuando ya no es necesario seguir analizando su ejecución interna.



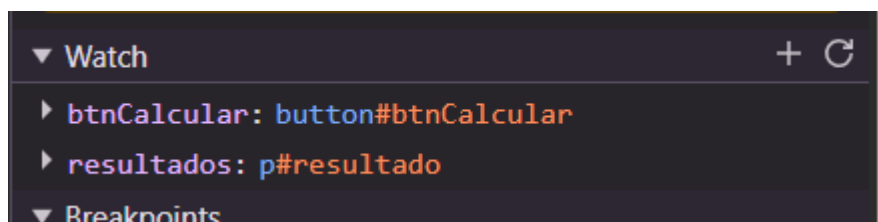
5.3 Panel Scope

El panel **Scope** muestra todas las variables accesibles en el momento en que la ejecución está detenida.



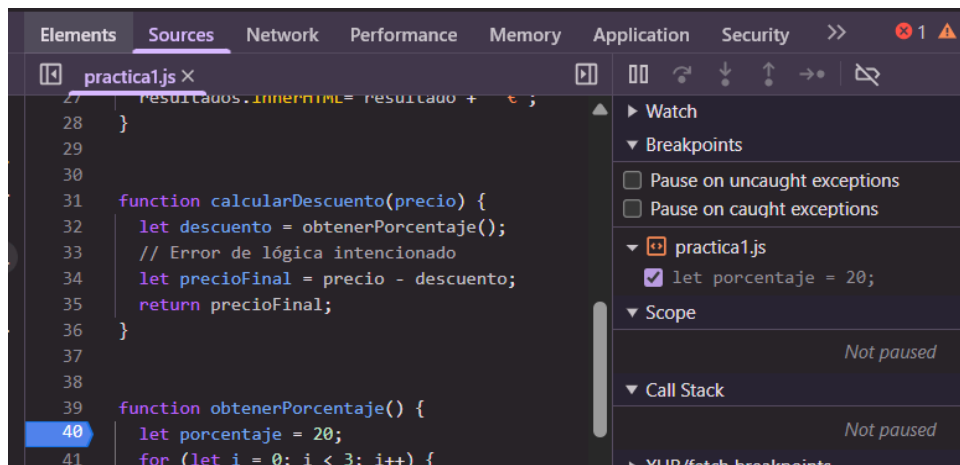
5.4 Watch expressions

El panel **Watch** permite monitorizar expresiones o variables concretas durante la ejecución del programa.



5.5 Pausa en excepciones

La opción **Pause on Exceptions** permite detener automáticamente la ejecución cuando se produce una excepción.



6.Pestaña Network

La pestaña **Network** permite monitorizar todas las solicitudes realizadas por una página web durante su carga y funcionamiento. Gracias a esta herramienta es posible analizar los recursos descargados, los tiempos de carga, las cabeceras HTTP y las respuestas recibidas por el navegador.

Para visualizar correctamente la actividad de red, se recomienda abrir la pestaña **Network** y recargar la página utilizando la tecla **F5**.

6.1 Carga de recursos

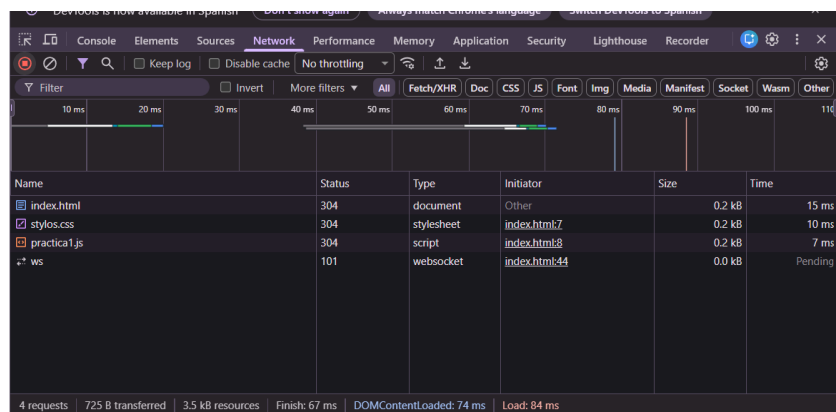
Cada vez que una página web se carga, el navegador descarga los recursos necesarios para mostrarla correctamente. En nuestra aplicación estos recursos son principalmente:

- El documento HTML.
- El archivo CSS.
- El archivo JavaScript.

Calculadora de descuento

Resultado: -

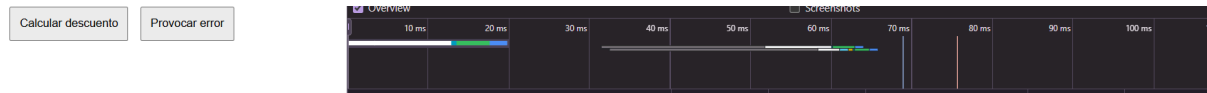
Calcular descuento Provocar error



6.2 Tiempos y waterfall

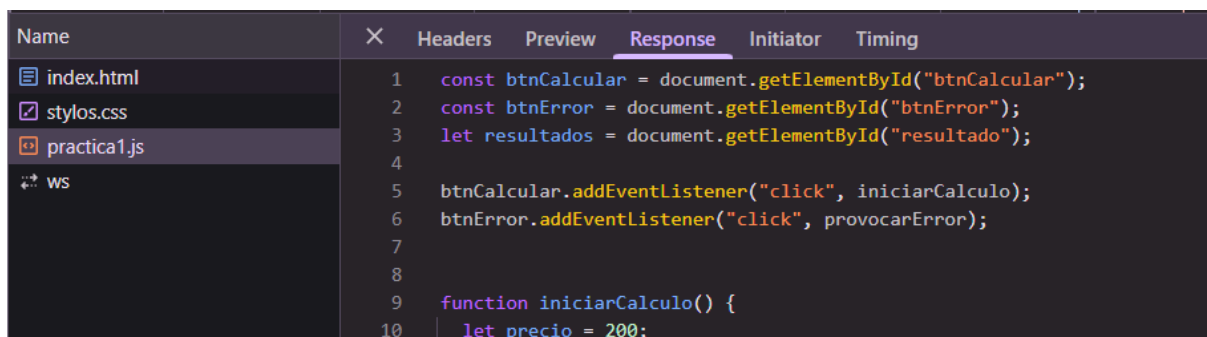
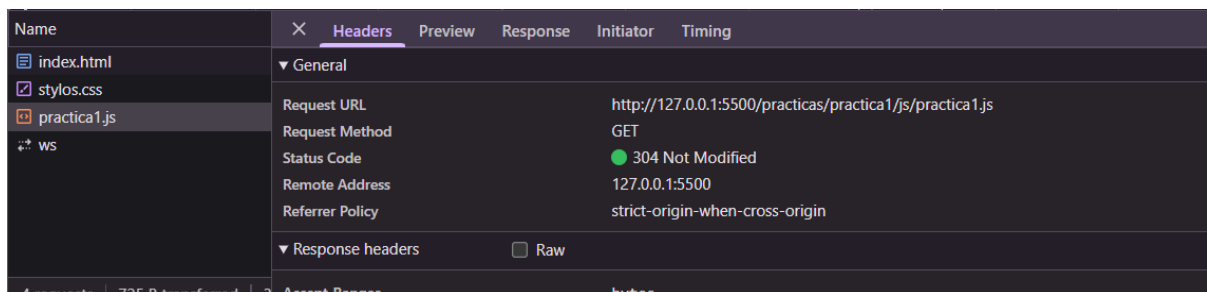
Una de las funciones más importantes de **Network** es el análisis de los tiempos de carga. DevTools representa gráficamente estos tiempos mediante el diagrama denominado **Waterfall**.

Cada barra muestra cuánto tiempo ha necesitado un recurso para descargarse.



6.3 Cabeceras y respuestas

Además de los tiempos de carga, Network permite inspeccionar la información intercambiada entre el navegador y el servidor mediante las cabeceras HTTP y las respuestas recibidas.



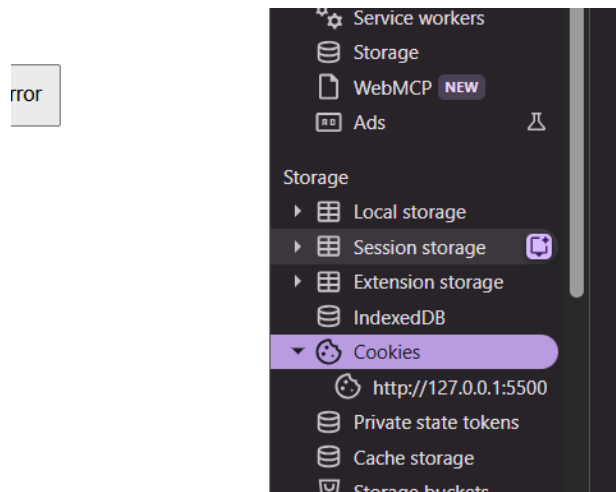
7.Pestaña Application

La pestaña **Application** permite gestionar y analizar la información que el navegador almacena localmente. Desde esta herramienta es posible consultar cookies, datos almacenados en LocalStorage, SessionStorage, bases de datos locales y otros mecanismos de almacenamiento utilizados por las aplicaciones web.

7.1 Cookies

Las **cookies** son pequeños fragmentos de información que una página web almacena en el navegador del usuario. Se utilizan habitualmente para mantener sesiones iniciadas, recordar preferencias o guardar información temporal.

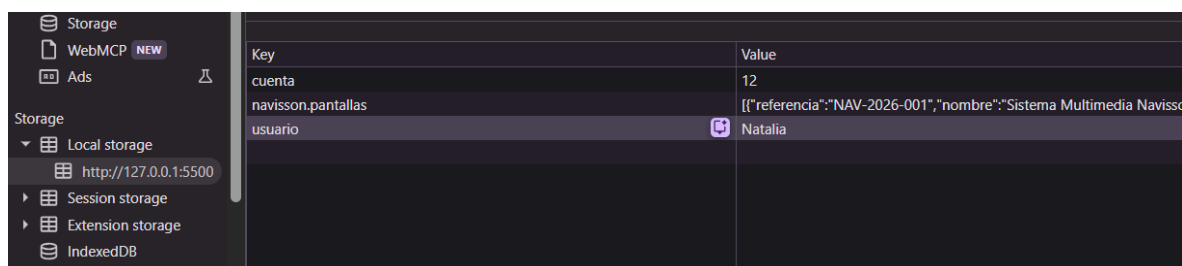
En la aplicación desarrollada para esta práctica no se utilizan cookies propias, pero la pestaña Application permite visualizar todas las cookies asociadas al sitio web actual.



7.2 LocalStorage

El **LocalStorage** permite almacenar información en el navegador de forma persistente. Los datos permanecen almacenados incluso después de cerrar el navegador o reiniciar el equipo.

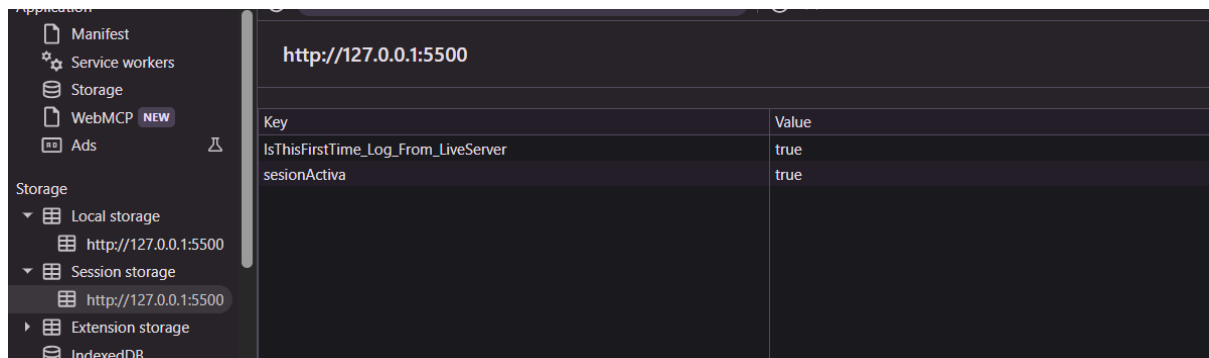
Aunque nuestra aplicación no utiliza LocalStorage de forma activa, DevTools permite acceder a esta sección para visualizar o modificar los datos almacenados.



7.3 SessionStorage

El **SessionStorage** funciona de forma similar a **LocalStorage**, pero los datos únicamente permanecen disponibles mientras la pestaña o ventana actual siga abierta.

Cuando la sesión finaliza y la pestaña se cierra, la información almacenada desaparece automáticamente.



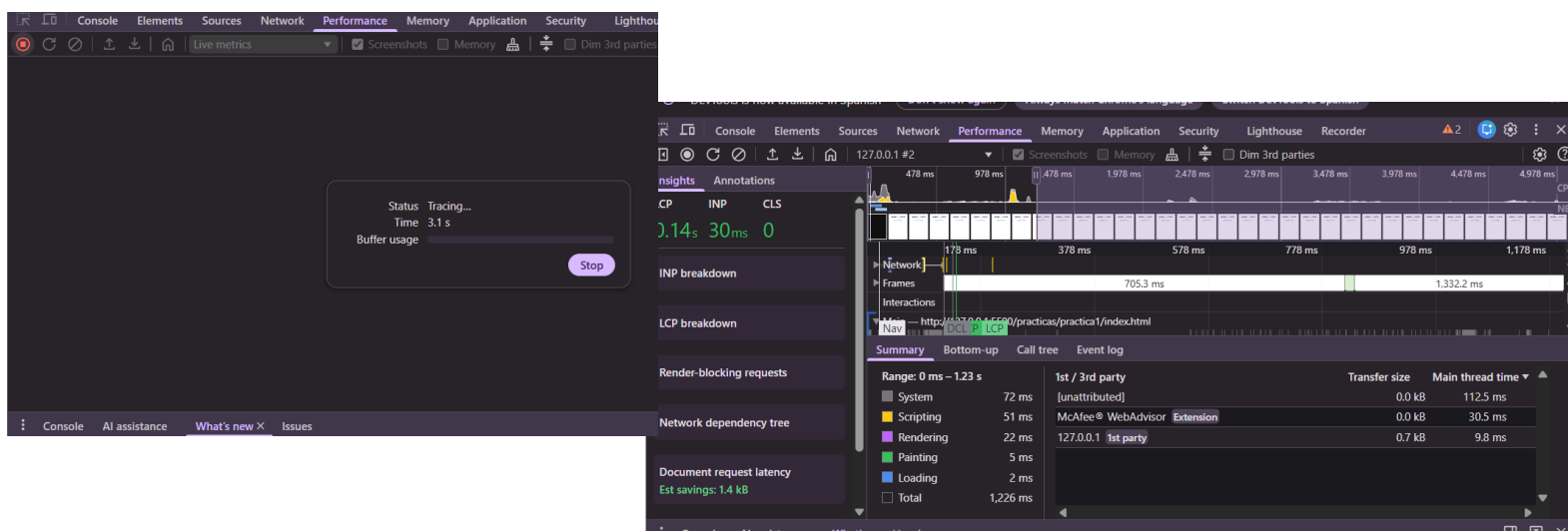
8.Pestaña Performance

La pestaña **Performance** permite analizar el comportamiento y el rendimiento de una página web mientras se ejecuta. Esta herramienta registra la actividad del navegador y muestra información relacionada con la ejecución de JavaScript, el renderizado de la página y los tiempos empleados en cada operación.

8.1 Grabación de rendimiento

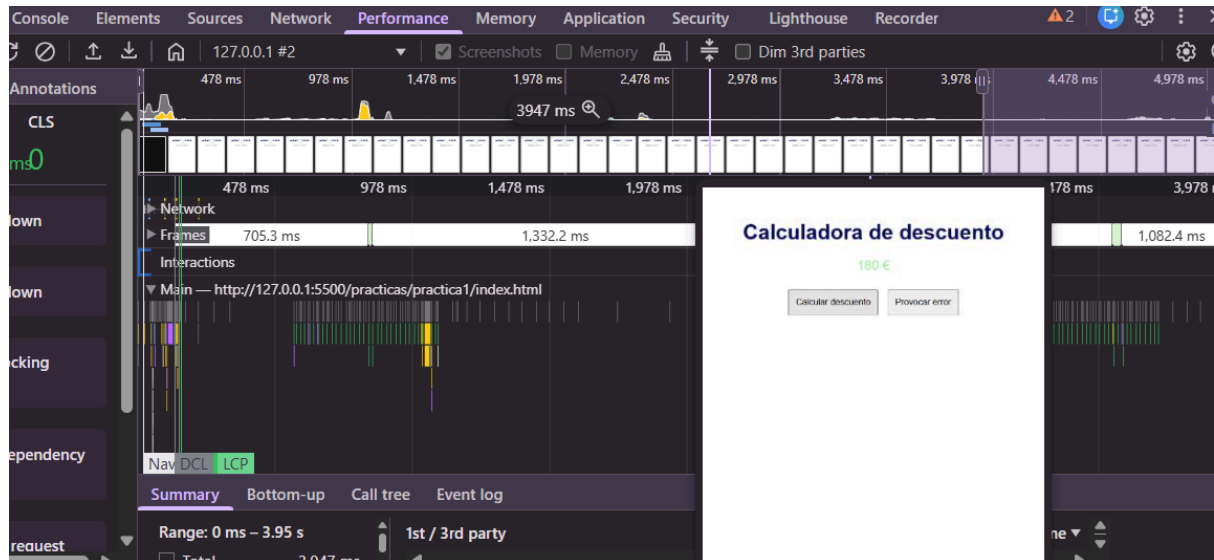
Performance permite registrar toda la actividad que se produce durante una interacción con la aplicación.

La grabación permite observar cómo afectan las acciones del usuario al rendimiento de la aplicación.



8.2 Interpretación de métricas

Una vez realizada la grabación, DevTools proporciona diversas métricas y gráficos que ayudan a comprender el comportamiento de la página.



9. Pestaña Lighthouse

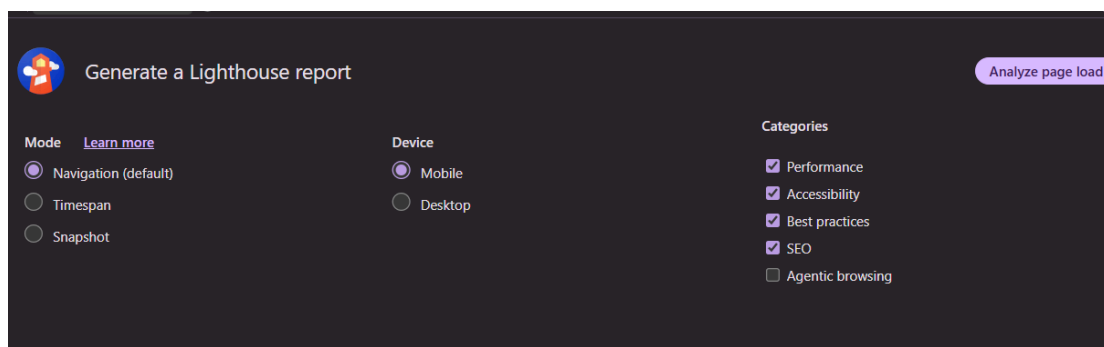
Lighthouse es una herramienta integrada en las DevTools que permite realizar auditorías automáticas sobre una página web. Estas auditorías analizan diferentes aspectos del sitio, como el rendimiento, la accesibilidad, las buenas prácticas de desarrollo y otros indicadores de calidad.

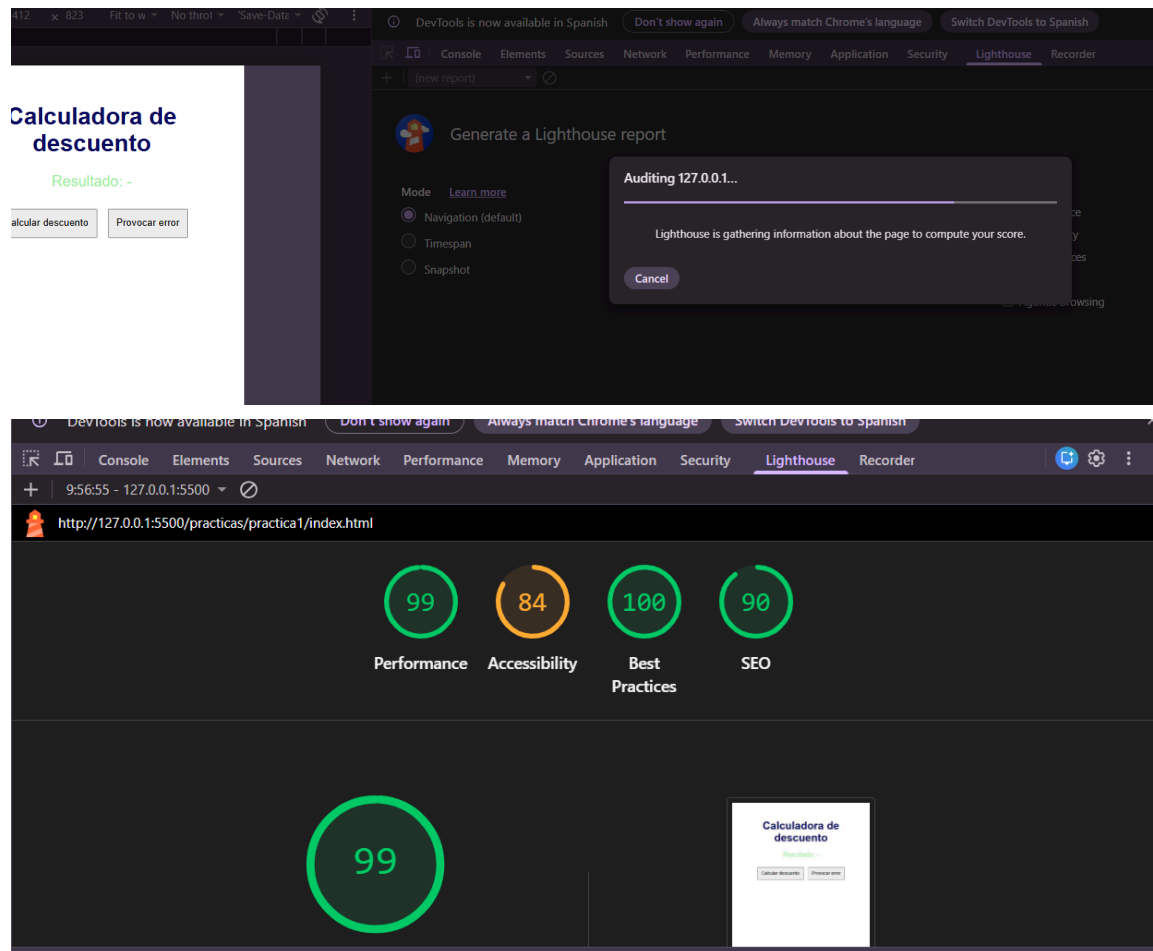
9.1 Auditoría

Para realizar una auditoría es necesario acceder a la pestaña **Lighthouse** desde las DevTools y generar un informe sobre la página desarrollada para la práctica.

La auditoría se realiza de forma automática y proporciona una evaluación objetiva de la página web.

Una vez finalizado el análisis, Lighthouse genera un informe con diferentes puntuaciones y recomendaciones organizadas por categorías.

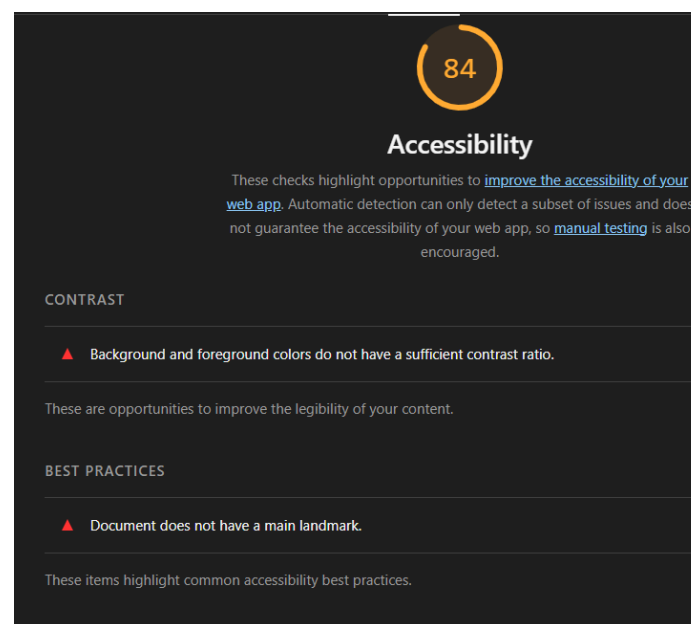
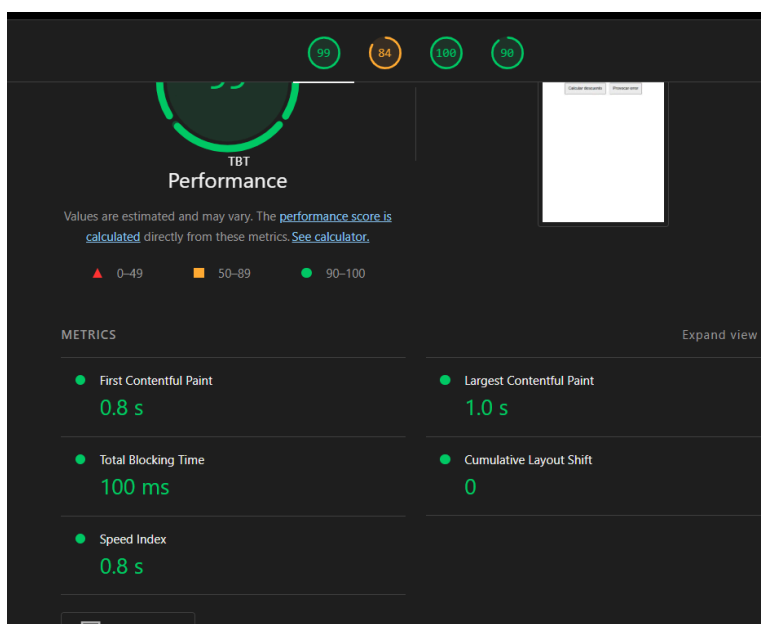




9.2 Resultados y recomendaciones

Tras la generación del informe, **Lighthouse** proporciona información detallada sobre los distintos aspectos analizados.

La métrica **Performance** permite valorar el rendimiento general de la aplicación y detectar posibles mejoras relacionadas con la carga y ejecución.



10.Conclusiones

Gracias a esta práctica he aprendido a utilizar las principales herramientas de las DevTools del navegador para inspeccionar elementos, depurar código JavaScript, analizar recursos y comprobar el rendimiento de una página web.

La pestaña **Sources** ha sido especialmente útil para comprender el funcionamiento del código mediante breakpoints y ejecución paso a paso, mientras que herramientas como **Console**, **Network** y **Application** facilitan la detección de errores y el análisis del comportamiento de una aplicación.

En conclusión, las DevTools son una herramienta fundamental para el desarrollo web, ya que permiten trabajar de forma más eficiente y solucionar problemas de manera rápida y organizada.

11.ENLACE A GITHUB

<https://github.com/Natalialuque>